Analisi del database Chinook_MySql

Tecniche di Programmazione

3 novembre 2026

Introduzione

Il database **Chinook** rappresenta un negozio digitale di musica (sulla falsariga di un vecchio iTunes Store): clienti che acquistano brani musicali tramite fatture, organizzati in album, artisti, generi e playlist. È composto da **11 tabelle**, qui raggruppate in tre aree logiche:

1. **Catalogo musicale**: `Artist`, `Album`, `Track`, `Genre`, `MediaType`
2. **Playlist**: `Playlist`, `PlaylistTrack` (tabella ponte)
3. **Vendite**: `Customer`, `Invoice`, `InvoiceLine`, `Employee`

Rispetto a IMDB_2, Chinook ha uno schema più "a catena" (quasi tutte le tabelle sono collegate in un'unica lunga sequenza di relazioni) e usa colonne in **CamelCase** (es. `CustomerId`, non `customer_id`): un dettaglio da ricordare sia scrivendo le query SQL sia definendo i campi delle dataclass, se si vuole usare la scorciatoia `Classe(**row)`.

Le tabelle

Artist

Colonna	Tipo	Note
ArtistId	INT	**PK**
Name	NVARCHAR(120)	nome dell'artista/gruppo

Tabella radice del catalogo: un artista può avere zero o più album. Non ha chiavi esterne proprie.

Album

Colonna	Tipo	Note
AlbumId	INT	**PK**
Title	NVARCHAR(160)	titolo dell'album
ArtistId	INT	**FK** → `Artist.ArtistId`, NOT NULL

Ogni album appartiene a **esattamente un** artista (relazione 1 a N tra Artist e Album: un artista può avere molti album, ma ogni album ha un solo artista).

Track

Colonna	Tipo	Note
TrackId	INT	**PK**
Name	NVARCHAR(200)	titolo del brano
AlbumId	INT	**FK** → `Album.AlbumId`, **NULLABLE**
MediaTypeId	INT	**FK** → `MediaType.MediaTypeId`, NOT NULL
GenreId	INT	**FK** → `Genre.GenreId`, **NULLABLE**
Composer	NVARCHAR(220)	nullable
Milliseconds	INT	durata in millisecondi
Bytes	INT	dimensione del file
UnitPrice	NUMERIC(10,2)	prezzo unitario di vendita

Il brano è l'entità più "centrale" dello schema: è collegata sia al catalogo (Album → Artist, Genre, MediaType) sia alle vendite (InvoiceLine) sia alle playlist (PlaylistTrack). **Attenzione**: `AlbumId` e `GenreId` sono nullable — un brano può non appartenere a nessun album o non avere un genere assegnato; nelle query che attraversano queste relazioni con una JOIN implicita (`FROM a, b WHERE a.x = b.y`) un valore NULL fa semplicemente "perdere" quella riga, come una INNER JOIN; se il testo dell'esame chiede di considerare *anche* i brani senza genere/album, serve una `LEFT JOIN` esplicita.

Genre

Colonna	Tipo	Note
GenreId	INT	**PK**
Name	NVARCHAR(120)	es. “Rock”

MediaType

Colonna	Tipo	Note
MediaTypeId	INT	**PK**
Name	NVARCHAR(120)	es. “MPEG audio file”, “AAC audio...”

Entità di scarso interesse per i grafi (compare raramente nei testi d’esame visti), ma utile per filtrare i brani per formato.

Playlist

Colonna	Tipo	Note
PlaylistId	INT	**PK**
Name	NVARCHAR(120)	es. “Music”, “Movies”

PlaylistTrack (tabella ponte)

Colonna	Tipo	Note
PlaylistId	INT	**FK** → `Playlist.PlaylistId`, **PK composta**
TrackId	INT	**FK** → `Track.TrackId`, **PK composta**

Realizza la relazione **molti a molti** tra `Playlist` e `Track`: una playlist contiene molti brani, e uno stesso brano può comparire in più playlist. Non ha altre colonne oltre alla chiave composta: è una tabella ponte “pura”.

Customer

Colonna	Tipo	Note
CustomerId	INT	**PK**
FirstName	NVARCHAR(40)	
LastName	NVARCHAR(20)	
Company	NVARCHAR(80)	nullable
Address	NVARCHAR(70)	nullable
City	NVARCHAR(40)	nullable
State	NVARCHAR(40)	nullable
Country	NVARCHAR(40)	nullable
PostalCode	NVARCHAR(10)	nullable
Phone	NVARCHAR(24)	nullable
Fax	NVARCHAR(24)	nullable
Email	NVARCHAR(60)	NOT NULL

Colonna	Tipo	Note
SupportRepId	INT	**FK** → `Employee.EmployeeId`, nullable

Ogni cliente è (opzionalmente) seguito da un dipendente (il suo "referente commerciale"). `Country`/`State`/`City` sono le colonne più usate per i filtri da dropdown nei testi d'esame (es. "l'utente seleziona un paese").

Invoice

Colonna	Tipo	Note
InvoiceId	INT	**PK**
CustomerId	INT	**FK** → `Customer.CustomerId`, NOT NULL
InvoiceDate	DATETIME	
BillingAddress/City/State/Country/PostalCode	NVARCHAR	dati di fatturazione, nullable
Total	NUMERIC(10,2)	totale della fattura

Rappresenta **un acquisto** (uno scontrino): un cliente può avere molte fatture. Il campo `Total` è già precalcolato (somma delle righe), quindi spesso NON serve ricalcolarlo da `InvoiceLine` per ottenere il fatturato di un cliente — basta `SUM(Invoice.Total)` raggruppato per cliente (si veda la sezione query più sotto).

InvoiceLine

Colonna	Tipo	Note
InvoiceLineId	INT	**PK**
InvoiceId	INT	**FK** → `Invoice.InvoiceId`, NOT NULL
TrackId	INT	**FK** → `Track.TrackId`, NOT NULL
UnitPrice	NUMERIC(10,2)	prezzo pagato per quel brano in quella riga
Quantity	INT	quantità (quasi sempre 1, per la musica)

È la tabella **ponte tra le vendite e il catalogo**: ogni riga dice "in questa fattura, è stato acquistato questo brano". È il punto di passaggio obbligato per collegare un cliente a un artista, un genere o un album acquistato (`Customer → Invoice → InvoiceLine → Track → Album → Artist`).

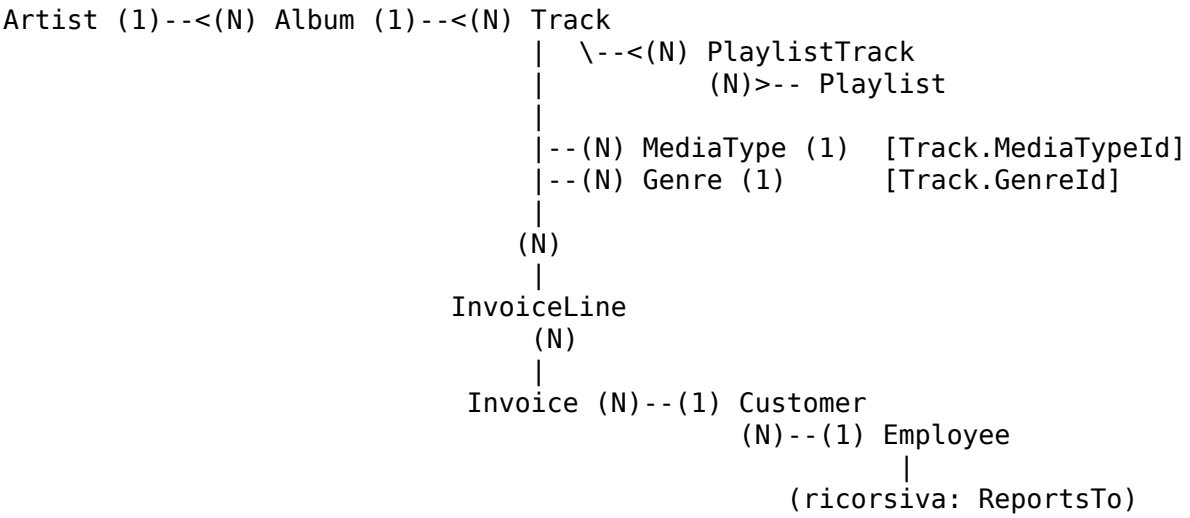
Employee

Colonna	Tipo	Note
EmployeeId	INT	**PK**
LastName/FirstName	NVARCHAR	nome del dipendente
Title	NVARCHAR(30)	ruolo (es. "Sales Support Agent")
ReportsTo	INT	**FK** → `Employee.EmployeeId` (ricorsiva!), nullable
BirthDate, HireDate	DATETIME	date, nullable

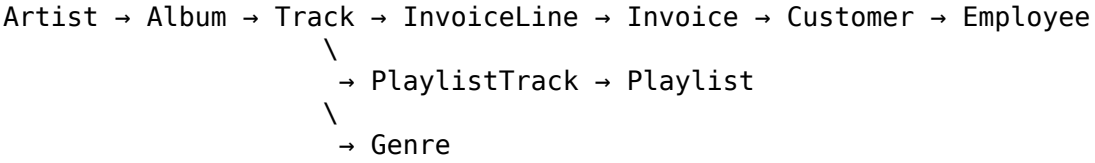
Colonna	Tipo	Note
Address, City, State, Country, PostalCode, Phone, Fax, Email	NVARCHAR	dati anagrafici, nullable

Nota particolare: `ReportsTo` è una FK che punta alla STESSA tabella `Employee` — rappresenta la gerarchia aziendale (ogni dipendente ha un responsabile, che è a sua volta un dipendente). Non comparendo in nessuno dei quattro esami analizzati, è probabile che — se richiesta — dia luogo a un grafo “organizzativo” ad albero, analogo per struttura ai grafi gerarchici già visti in altri corsi.

Schema delle relazioni



In sintesi, la "catena" centrale che va dal catalogo musicale alle vendite è:



Questa catena è la base di quasi tutte le query d'esame: per collegare un cliente a un artista (o viceversa) bisogna attraversare **5 tabelle** (`Customer`, `Invoice`, `InvoiceLine`, `Track`, `Album` prima di arrivare ad `Artist`); per collegare un artista a una playlist, altre **3** (`Album`, `Track`, `PlaylistTrack` prima di `Playlist`).

Possibili query d'esame

Di seguito gli schemi di query più probabili, ricavati dall'analisi di quattro tracce d'esame reali che usano questo database. In tutte si preferisce lo stile "virgola nel FROM + condizioni nel WHERE" (equivalente ai classici `JOIN ... ON`, si veda il vademecum per il confronto).

1. Valori per popolare una dropdown

```sql
SELECT DISTINCT c.Country
FROM Customer c
WHERE c.Country IS NOT NULL
ORDER BY c.Country;
```

Pattern analogo per generi (`Genre.Name`), stati (`Customer.State`), ecc.

2. Nodi con un attributo numerico calcolato (es. fatturato di un cliente)

```sql
SELECT c.*, SUM(i.Total) AS Fatturato
FROM Customer c, Invoice i
WHERE c.CustomerId = i.CustomerId
AND c.Country = %s
GROUP BY c.CustomerId
ORDER BY c.CustomerId;
```

Da notare: `GROUP BY c.CustomerId` è necessario perché nella SELECT compare una funzione di aggregazione (`SUM`); `c.*` prende tutte le colonne di `Customer`, comode da passare con `Customer(**row)` se la dataclass ha gli stessi nomi di campo.

3. Popolarità complessiva di un artista (somma su tutti i clienti)

```sql
SELECT i.CustomerId, art.ArtistId, COUNT(*) AS NumBrani
FROM Invoice i, InvoiceLine il, Track t, Genre g, Artist art, Album al
WHERE i.InvoiceId = il.InvoiceId
AND t.TrackId = il.TrackId
AND t.AlbumId = al.AlbumId
AND g.GenreId = t.GenreId
AND art.ArtistId = al.ArtistId
AND g.Name = %s
GROUP BY i.CustomerId, art.ArtistId;
```

Restituisce una terna `(cliente, artista, numero brani acquistati)` per ogni combinazione, da raggruppare poi in Python con un `defaultdict(dict)` per cliente, e usare `itertools.combinations` per generare le coppie di artisti acquistati insieme da uno stesso cliente (pattern osservato in una delle soluzioni analizzate: utile per costruire archi "C1→C2 se condividono un acquirente", pesati sulla popolarità complessiva).

4. Coppie (nodo, correlato) per calcolare intersezioni tra nodi

```sql
-- quali artisti ha acquistato ciascun cliente di un certo paese
SELECT DISTINCT c.CustomerId, art.ArtistId
FROM Customer c, Invoice i, InvoiceLine il, Track t, Album al, Artist art
WHERE c.CustomerId = i.CustomerId
AND i.InvoiceId = il.InvoiceId
```

```
AND il.TrackId = t.TrackId
AND t.AlbumId = al.AlbumId
AND al.ArtistId = art.ArtistId
AND c.Country = %s;
```

Il risultato va raggruppato in Python in un dizionario `{CustomerId: [ArtistId, ...]}`: nel model si trasformano le liste in insiemi e si calcola l'intersezione per decidere se due clienti sono collegati (hanno comprato almeno un artista in comune).

5. Coppie (artista, playlist) attraverso una catena a 3 JOIN

```
SELECT DISTINCT art.ArtistId, pt.PlaylistId
FROM Artist art, Album al, Track t, PlaylistTrack pt
WHERE art.ArtistId = al.ArtistId
AND al.AlbumId = t.AlbumId
AND t.TrackId = pt.TrackId;
```

Stesso pattern della query precedente, utile quando il criterio di collegamento tra due nodi passa per le playlist condivise (peso dell'arco = numero di playlist in comune).

6. Dettaglio di un singolo nodo (query "per nodo", chiamata in un ciclo)

```
-- tutte le tracce di un album
SELECT t.*
FROM Track t
WHERE t.AlbumId = %s
ORDER BY t.Name;
```

```
-- solo il conteggio, quando non servono i dettagli (es. per mostrare
-- "numero album" di un artista solo in fase di stampa finale)
SELECT COUNT(DISTINCT al.AlbumId) AS NumAlbum
FROM Album al
WHERE al.ArtistId = %s;
```

7. Filtro su genere/album con almeno un brano (evitare nodi "vuoti")

```
SELECT DISTINCT a.*
FROM Album a, Track t
WHERE a.AlbumId = t.AlbumId
ORDER BY a.Title;
```

Nota il `DISTINCT a.*` (non si usano funzioni di aggregazione: serve solo a evitare righe duplicate quando un album ha più tracce). Pattern analogo per "artisti con almeno un brano in un dato genere":

```
SELECT DISTINCT art.ArtistId AS ArtistID, art.Name AS Name
FROM Artist art, Album al, Track t, Genre g
WHERE art.ArtistId = al.ArtistId
AND al.AlbumId = t.AlbumId
AND t.GenreId = g.GenreId
AND g.Name = %s;
```

(Gli alias `AS ArtistID, AS Name` servono solo se i nomi dei campi della dataclass non coincidono esattamente col nome della colonna originale.)

8. Query aggregate “al volo” per la stampa finale

```sql
-- quanti brani ha acquistato un cliente specifico, per un dato genere
SELECT COUNT(*) AS NumBrani
FROM Invoice i, InvoiceLine il, Track t, Genre g
WHERE i.InvoiceId = il.InvoiceId
AND il.TrackId = t.TrackId
AND t.GenreId = g.GenreId
AND i.CustomerId = %s
AND g.Name = %s;
```

Utile quando il testo chiede un dato puntuale su un nodo già scelto, non durante la costruzione del grafo ma solo al momento di stamparne i dettagli (si usa `cursor.fetchone()` dato che il risultato è una riga singola).

In sintesi

I criteri “tipo di arco” più ricorrenti nei testi d’esame su Chinook sono:

- **nodo = cliente**, arco se due clienti hanno **comprato almeno un artista in comune** (peso/verso dal fatturato);
- **nodo = album**, arco se due album **condividono almeno un genere** tra le rispettive tracce (non pesato);
- **nodo = artista**, arco se due artisti **compaiono in almeno una playlist comune** (peso = numero di playlist condivise);
- **nodo = artista**, arco se **un cliente ha comprato brani di entrambi** (peso/verso dalla popolarità complessiva, costruito con `itertools.combinations` per cliente).

In tutti i casi, il filtro iniziale (paese del cliente, genere dell’artista) serve a limitare il grafo a un sottoinsieme gestibile di nodi, scelto dall’utente tramite una dropdown.